

representation jointly with the rest of the model. We used the embedding of the [CLS] token as the input to the linear layer for prediction.

3.2.4 Label Scaling

For regression tasks, we train LLM-Prop on normalized targets using different label normalization techniques: z-score normalization, min-max normalization and log normalization. Then we denormalize the outputs to calculate the prediction errors on the actual values. Suppose that we have N crystals $X_1, \dots, X_N$ and their corresponding properties $Y_1, \dots, Y_N$ as a training set. Here X_i denotes the text description of the i^{th} crystal. Denote by μ and σ the mean and standard deviation over all targets in the training set, respectively. Denote by $Y_{\min}$ and $Y_{\max}$ the minimum and the maximum target value in the training data, respectively. The three normalization techniques we explored normalize a given target value Y_i as follows:

$$\hat{Y}_i(\text{z-score}) = \frac{Y_i - \mu}{\sigma}, \quad \hat{Y}_i(\text{min-max}) = \frac{Y_i - Y_{\min}}{Y_{\max} - Y_{\min}}, \quad \hat{Y}_i(\text{log-norm}) = \log Y_i + 1.$$

4 Experiments

We compare LLM-Prop with four baselines: three GNN-based models—CGCNN (Xie and Grossman, 2018), MEGNet (Chen et al., 2019), and ALIGNN (Choudhary and DeCost, 2021)—and one text-based model, MatBERT (Walker et al., 2021). The GNN-based methods are also trained and evaluated on the same benchmark dataset using the same splits to ensure a fair comparison. Note we do not include the results reported in the original articles since the dataset used was collected from an old version of the Materials Project, the 2018 version, and only contains about 70k crystals. We train all models using NVIDIA RTX A6000 GPUs. We next describe how we set up the experiments for all the baselines and for LLM-Prop.

4.1 Baselines

CGCNN. We directly use the publicly available code⁴ and the same experimental configurations as reported in the original paper to train a CGCNN from scratch and evaluate it on our benchmark dataset.

MEGNet. We train MEGNet from scratch and evaluate it on our benchmark dataset following the same implementation details and configurations described in the original article using the publicly available code released by the authors⁵. Unfortunately, for MEGNet, we could not find the implementation details on classification tasks, thus we only compared against it on regression tasks.

ALIGNN. Similarly to other baselines, we follow the original implementation details of ALIGNN released in the paper using the publicly available code from the authors⁶ to train and evaluate it on our benchmark dataset.

⁴<https://github.com/txie-93/cgcnn>

⁵<https://github.com/materialsvirtuallab/megnet>

⁶<https://github.com/usnistgov/alignn>